Về việc đăng ký OSF khi một phần Tầng 1 đã chạy trên test

Câu trả lời ngắn: không cần chạy lại test của các arm Tầng 1, chỉ để lượt chạy có ngày sau khi đăng ký. Nếu dự đoán và điểm số đã lưu đầy đủ, được tạo bằng đúng dữ liệu, prompt, model và cấu hình sẽ dùng cho lượt cuối, thì giữ nguyên các đầu ra đó và tính lại chỉ số bằng script đánh giá thống nhất. Nhưng cũng phải nói rõ chiều ngược lại: chạy lại trên cùng tập test không biến kết quả đã biết thành kết quả "đăng ký trước", cũng không trở thành một phép xác nhận độc lập. Đăng ký trước ràng buộc những việc sẽ làm tiếp theo và buộc mình khai báo trung thực những gì đã biết; nó không xóa được việc đã tiếp xúc với test. Có 3 việc cần tách bạch và một danh sách việc cần làm ngay.

1. Kết quả đã biết trước khi đăng ký phải được khai báo là đã biết

- Lập một bảng kê tiếp xúc (mẫu đã có trong configs/protocol.yaml, khối exposure_ledger): mỗi dòng là một arm đã chạy — dataset, split, ngày chạy đầu tiên, commit, đã xem những gì (chỉ đầu ra model / MCC, AUROC / so sánh giữa các arm), manifest có khớp giao thức sẽ khóa hay không, và hành động (giữ / chạy lại / chỉ chạy lại B2-rep).
- Trong mẫu đăng ký của OSF, mục "Existing data" phải chọn đúng mức: nếu đã tính MCC/AUROC của Tầng 1 trên test thì chọn "Registration following analysis of the data" (nêu rõ phạm vi: arm nào, dataset nào); nếu mới chỉ tạo đầu ra mà chưa tính chỉ số thì chọn "prior to analysis". Dán bảng kê vào phần giải thích. Các arm Tầng 2–3 và mọi phân tích chưa làm (Bảng B, Bảng D, routing, chấm mù) đăng ký như bình thường vì chưa chạy.
- Trong bài, Bảng B có cột trạng thái cho từng cặp so sánh: "đã xem trước đăng ký" hoặc "chưa xem". Những gì đã xem không được gọi là đăng ký trước, dù có gọi lại model sau ngày đăng ký.

2. Đăng ký quy tắc chọn ngưỡng — không phải một con số, cũng không phải một ngưỡng chung cho mọi arm

Hướng hai bạn đề xuất là đúng, chỉ cần phát biểu chính xác hơn: cái chung cho mọi tầng là *quy tắc*, còn giá trị thì riêng cho từng dataset, arm và seed:

$\tau^*(d, a, r) = \text{argmax over } \tau \in \{0{,}01; 0{,}02; \ldots; 0{,}99\} \text{ của } MCC(y_{val}(d), 1[s_{val}(d, a, r) \geq \tau])$

Khóa kèm theo quy tắc: danh sách mẫu validation (hash manifest), lưới ngưỡng, cách phá hòa (lấy ngưỡng gần 0,5 nhất), chỉ dùng mẫu hợp lệ, bỏ qua ngưỡng làm MCC không xác định, và quy tắc thừa kế: B2-rep và B2-flip dùng τ^* của B2, B3-flip và B3-swap dùng τ^* của B3 — chọn lại ngưỡng cho một arm đã bị đảo nhãn hay lặp sẽ "sửa" đúng hiệu ứng mình cần đo và trộn thêm biến động do chọn ngưỡng. Không thay đổi bất kỳ quy tắc nào trong số này sau khi đã xem test.

Không cần chạy hết validation của Tầng 2–3 trước khi đăng ký để có sẵn tất cả các giá trị. Đăng ký thuật toán, rồi với mỗi arm làm theo đúng thứ tự: validation → tính τ* → khóa (ghi file, băm, commit) → test. Với các arm Tầng 1 đã có validation, tính τ* ngay bây giờ từ các records validation hiện có và ghi luôn cả giá trị lẫn cách tính vào hồ sơ đăng ký.

3. Khi nào mới phải chạy lại

Chỉ ba trường hợp, và mỗi trường hợp chỉ chạy lại đúng arm bị ảnh hưởng:

- *Lượt cũ không khớp giao thức sẽ bị khóa: khác model/checkpoint, backend, tokenizer, prompt contract (thứ tự khối theo Hình 6), ngân sách token hoặc* specs. So manifest.json của lượt cũ với protocol.yaml trước khi tag; lệch một mục là chạy lại — vì cấu hình, không phải vì ngày tháng.
- *Lượt cũ không còn đủ điểm số*: nếu chỉ lưu nhãn 0/1 mà không có logprob đủ để khôi phục s thì không thể áp dụng ngưỡng mới hoặc tính AUROC một cách hợp lệ — phải bổ sung đầu ra và ghi rõ lý do. Nếu đã lưu dict top-k logprob thì tính lại s offline và kiểm tra tỷ lệ prompt có *cả hai* token 0 và 1 trong top-k. Với token vắng mặt, dùng cận trên p ≤ p(token thấp nhất trong top-k) và gắn cờ s_bounded; không gán p = 0 (vắng trong top-5 không có nghĩa là xác suất bằng 0). Nếu tỷ lệ bị chặn vượt quá 1%, lượt đó cần chạy lại với top_logprobs = 20 (vLLM: --max-logprobs 20).
- *B2-rep từng được phục vụ từ cache*: B2-rep được dựng cùng prompt với B2, nên runner bản cũ có thể đã trả lại câu trả lời từ cache của B2 — một lượt "lặp" như vậy không đo được gì. Kiểm tra cờ cached (hoặc độ trễ 0 và trùng tuyệt đối từng logprob với B2); nếu đúng, chỉ chạy lại B2-rep.

Giữ nguyên timestamp và manifest của mọi lượt cũ; tạo một phiên phân tích mới tham chiếu đến chúng thay vì sửa hồ sơ cũ thành lượt đăng ký sau.

4. Gói mã đã được cập nhật cho đúng các quy tắc trên (grace_diag_kit bản 1.1, xem CHANGELOG.md)

- grace_diag.thresholds (mới): tính τ* chỉ từ records validation (+ anchor_scores.csv), áp dụng quy tắc thừa kế threshold_from trong arms.yaml, ghi runs/thresholds.json kèm sha256 của mọi đầu vào và của chính nội dung; khi nội dung thay đổi, sẽ sao lưu bản cũ và cảnh báo.
- evaluate_final.py nay bắt buộc --thresholds, chỉ đọc file đã khóa, báo lỗi nếu một arm có test mà không có ngưỡng (không còn mặc định 0,5, không tính ngưỡng ở bước này nữa).
- B2-rep (repeat: true) luôn gọi model mới với khóa cache riêng; records được ghi vào cache_tag.

- parse_top_logprobs trả thêm both_in_topk, s_bounded; top_logprobs mặc định là 20; smoke test chỉ đạt khi ≥ 99% prompt có cả hai token và ≥ 98% trùng nhãn giữa 2 lần gọi.

Thứ tự lệnh cho mỗi dataset: run_matrix.py --split validation → python -m grace_diag.thresholds --runs runs/final --arms configs/arms.yaml --anchor-csv … --out runs/thresholds.json → commit → run_matrix.py --split test → python -m grace_diag.evaluate_final … --thresholds runs/thresholds.json.

Việc cần làm ngay, theo thứ tự

1. Lập bảng kê các lần tiếp xúc cho từng arm đã chạy (validation và test).
2. Với từng lượt cũ, kiểm tra 3 điều kiện ở mục 3; ghi kết luận giữ/chạy lại vào bảng kê.
3. Cập nhật gói mã lên bản 1.1; tính và khóa τ* cho các arm Tầng 1 từ validation hiện có; commit thresholds.json.
4. Điền protocol.yaml (kể cả exposure_ledger, mức "Existing data") và preregistration.md, trong đó quy tắc ngưỡng được viết dưới dạng công thức như mục 2; tag prereg-v1; đăng lên OSF.
5. Chỉ sau đó mới chạy test cho các arm còn thiếu hoặc phải chạy lại, rồi Tầng 2–3 theo đúng thứ tự validation → khóa → test.

Trong bài viết, mô tả đúng bản chất: đây là các phân tích được chỉ định trước trên một benchmark đã có sẵn, không phải một nghiên cứu được thiết kế khi chưa biết gì về test; bằng chứng xác nhận độc lập dành cho tập đánh giá chưa tham gia vào việc chọn giả thuyết, cấu hình hay phương pháp (bản tạp chí). Đề cương đã được sửa ở Bước 4 và Mục 6.7 để phù hợp với tinh thần này.